

ИТМО · ТЕСТИРОВАНИЕ ПО · ОСЕНЬ 2026

Роль тестирования в жизненном цикле ПО

Модуль 1 · Лекция 2

Андрей Попов · Lekton

4 курс

ИТМО · Тестирование ПО

План лекции

01 Жизненный цикл ПО (SDLC):
модели и фазы

02 Где тестирование в каждой фазе

03 Процесс тестирования (STLC):
фазы и артефакты

04 Роли в тестировании: QA, test
engineer, разработчик, аналитик

05 Тестирование в enterprise vs
стартап

06 Модель данных СМП — основа
для тест-дизайна



Тестирование — не одна фаза в конце, а деятельность на ВСЕХ фазах

Качество строится на каждом этапе, а не «проверяется» post factum — ISTQB:
«Testing appears in each life cycle model, but with very different meanings».

Модели SDLC



Waterfall (каскадная)

Фазы строго последовательно, тестирование — в самом конце



V-модель

Каждой фазе разработки соответствует своя фаза тестирования



Agile / Scrum

Тестирование непрерывно, в каждом спринте

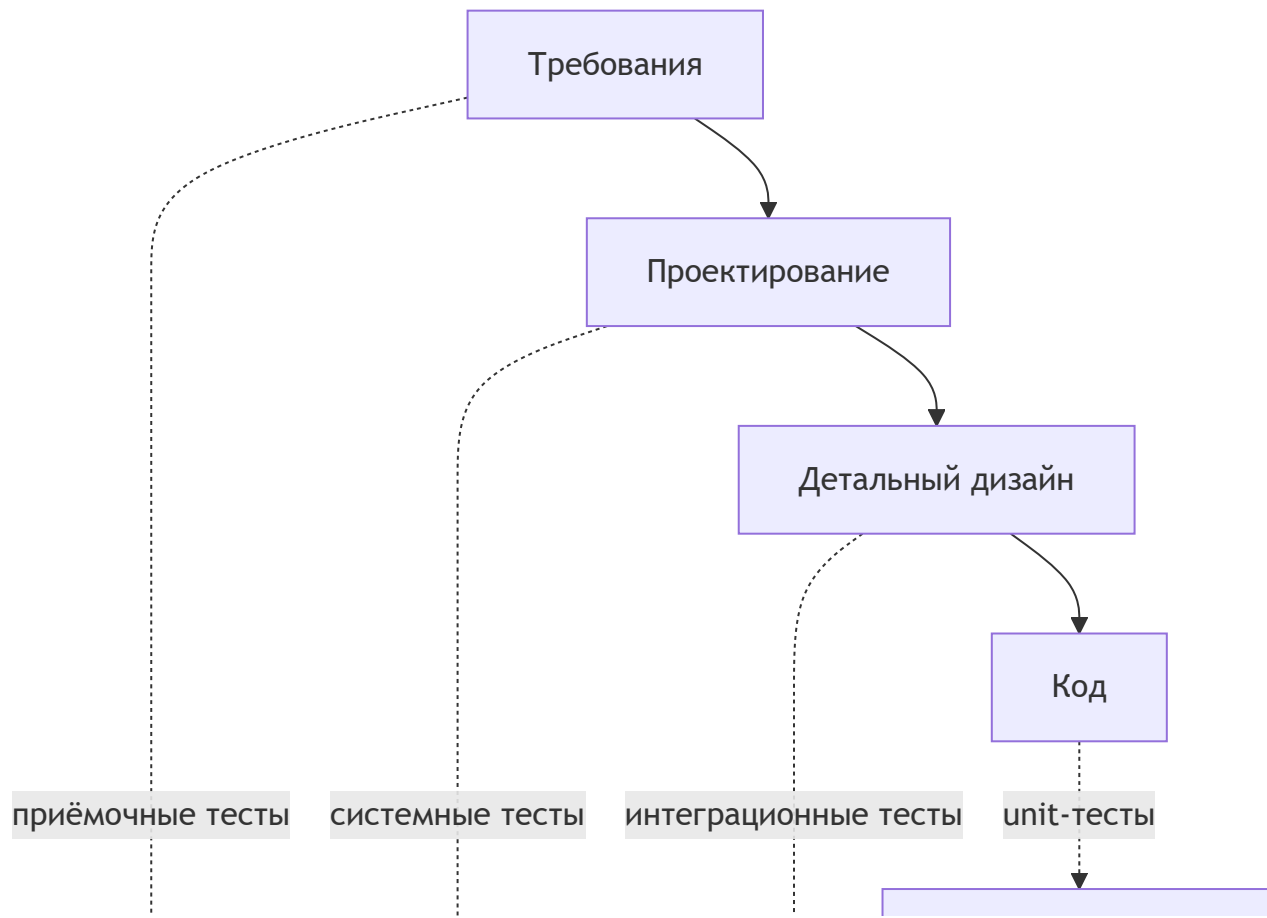


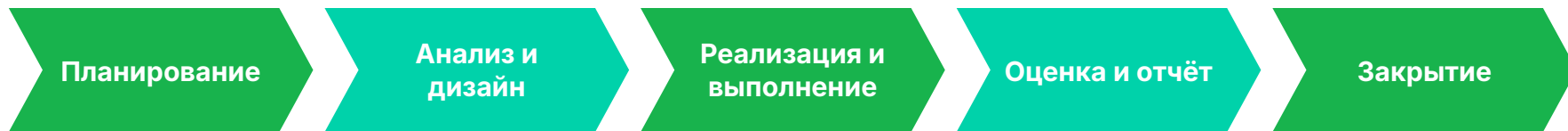
Гибрид (enterprise)

V-модель для регуляtorики + Agile для фич

Где тестирование в каждой фазе SDLC

Фаза	Деятельность тестировщика
Требования	Анализ тестируемости, выявление противоречий, приёмочные критерии
Проектирование	Ревью архитектуры, план тестирования, оценка рисков
Разработка	Unit-тесты, статический анализ кода, code review
Тестирование	Интеграционные, системные, E2E, регрессионные тесты
Развёртывание	Smoke-тесты, canary-релизы, мониторинг
Поддержка	Регрессия при багфиксах, тестирование патчей





STLC

Software Testing Life Cycle — процесс внутри процесса

Agile

STLC сжат до спринта, но фазы те же — быстрее и итеративнее

Артефакты тестирования



Тест-план

Что, как, когда и кем тестируется; критерии начала и окончания



Тест-кейс

Предусловия, шаги, входные данные, ожидаемый результат



Баг-репорт

Заголовок, шаги воспроизведения, ожидание vs факт, severity, priority



Тест-отчёт

Что проверено, что найдено, какие риски остались, рекомендации



Тестовая стратегия

Уровни, инструменты, автоматизация vs ручное

Роли в тестировании

QA Engineer

Процессы, стандарты, метрики качества

«Как мы обеспечиваем качество»

Уровень организации

Test Engineer

Проектирование и автоматизация тестов

«Как мы проверяем продукт»

Уровень продукта

Enterprise vs стартап



Enterprise

Долгоживущий код, много интеграций, строгая регуляторика → deep testing



Стартап

Быстрые итерации, минимум документации → shallow testing



Quality gates: например, coverage $\geq 70\%$, все тесты зелёные, нет критических багов.

В курсе: GitHub Actions · JaCoCo · Allure Reports

Реальность Lekton

Пирамида тестирования

Unit-тесты — разработчики

Модульные тесты — Java, автоматизированные

Интеграционные контура — малые

E2E + системное тестирование

Клиентские конфигурации

Практика

~40 тестировщиков в команде

CI/CD на всех контурах

Собственные инструменты нагрузочного тестирования

Java — основной язык модульных и E2E-тестов

Ручное тестирование — Support и Delivery

Card

```
pan · bin · status · dailyLimit ·  
monthlyLimit · availableBalance ·  
expiryDate
```

```
status: ACTIVE | INACTIVE | BLOCKED |  
EXPIRED
```

Transaction

```
mti · stan · rrn · pan · amount · status ·  
declineReason · authCode
```

```
status: APPROVED | DECLINED
```

Ключевые поля для тест-дизайна: **status** (4 значения), **limits** (daily/monthly), **amount**, **expiryDate** — основа классов эквивалентности, границ и decision tables.

Ключевые принципы архитектуры СМП



Только свои карты

Нет внешних BIN — упрощение для учебного проекта



Гибридное взаимодействие

Синхронный HTTP + асинхронный RabbitMQ



Eventual consistency

Запись в лог не мгновенно, а после доставки через очередь



Изоляция через API

Сервисы общаются через REST и очереди, а не через БД



Внешний API Bin Lookup

Timeout/retry — точка отказа



Каждый принцип

Порождает тестовые сценарии

Что будет на практике

Практика 1: запуск СМП + smoke-тесты → Артефакт 1 (6 баллов)

Практика 2: приём артефакта 1 + разбор архитектуры СМП, карта требований

ДЗ к модулю 2: изучить T3 Authorization и Card Management

Подготовка к артефакту 2 (тест-дизайн)

Классы эквивалентности · границы · decision table

Вопросы?